

TASK 21: #51625

1. host a simple php web application in the aws instance
2. configure an Application load balancer for the instance containing web application.
3. create an auto scaling group with scaling policies for the application instance.
4. attach the load balancer with auto scaling group.
5. verify the working of scaling policies by increasing the load manually

Step 1: Launch an EC2 instance with user data

- : the startup script installs Apache and PHP and creates the web page with the "Show Instance IP Address" button.

- AMI: Amazon Linux 2023
- Instance type: t2.micro or t3.micro
- Security group: web-sg
- User data:

```
#!/bin/bash

dnf update -y

dnf install -y httpd php8.2 stress-ng

systemctl enable --now httpd

cat > /var/www/html/index.php <<'EOF'
<?php
function imds($path) {
    $ctx = stream_context_create(['http' => [
        'method' => 'PUT',
        'header' => "X-aws-ec2-metadata-token-ttl-seconds: 60\r\n",
        'timeout' => 2
    ]]);
    $token = @file_get_contents('http://169.254.169.254/latest/api/token', false, $ctx);
    $ctx2 = stream_context_create(['http' => [
```

```

        'method' => 'GET',

        'header' => "X-aws-ec2-metadata-token: $token\r\n",

        'timeout' => 2

    ]]);

    $r = @file_get_contents("http://169.254.169.254/latest/meta-data/$path", false,
    $ctx2);

    return $r ?: 'N/A';
}

$privateIp = imds('local-ipv4');
$publicIp = imds('public-ipv4');
$instanceId = imds('instance-id');
$az = imds('placement/availability-zone');
?>

<!DOCTYPE html>

<html>

<head>

    <meta charset="UTF-8">

    <title>PHP App on AWS</title>

    <style>

        body { font-family: Arial, sans-serif; text-align: center; margin-top: 80px; }

        button { padding: 12px 28px; font-size: 16px; cursor: pointer; }

        #info { margin-top: 30px; font-size: 18px; display: none; }

    </style>

</head>

<body>

    <h1>Welcome to my PHP Web Application</h1>

    <button onclick="document.getElementById('info').style.display='block'">

        Show Instance IP Address

```

```

</button>

<div id="info">

  <p><b>Private IP:</b> <?= htmlspecialchars($privateIp) ?></p>

  <p><b>Public IP:</b> <?= htmlspecialchars($publicIp) ?></p>

  <p><b>Instance ID:</b> <?= htmlspecialchars($instanceId) ?></p>

  <p><b>Availability Zone:</b> <?= htmlspecialchars($az) ?></p>

</div>

</body>

</html>

EOF

```

```

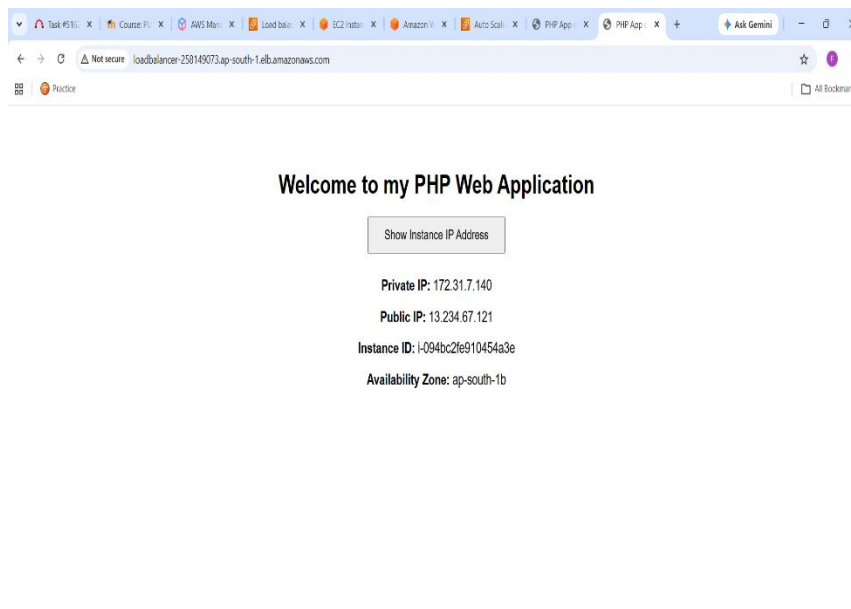
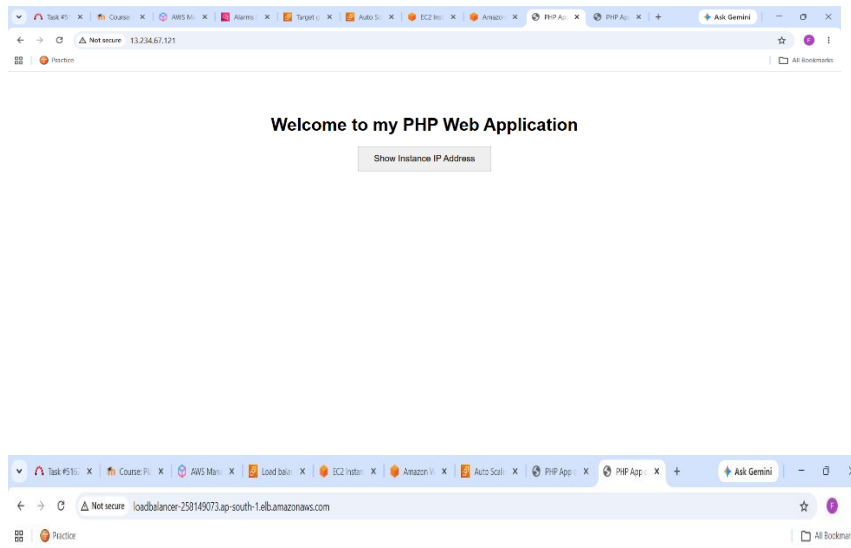
echo "OK" > /var/www/html/health.html

systemctl restart httpd

```

Step 2: Test the page

- : confirm the application runs and the button shows the instance IP address.



Part 2: Configure the Application Load Balancer

Step 3: Create the target group (tg1)

a list of instances that receive traffic, with a health check to skip unhealthy ones.

- Target type: Instances, HTTP, port 80
- Health check path: /health.html
- Register the existing instance

Step 5: Create the Application Load Balancer

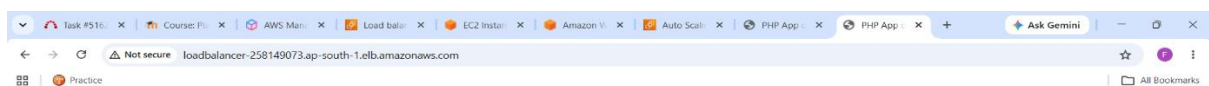
- : one public URL that spreads traffic across all healthy instances.

- Scheme: Internet-facing
- Subnets: at least 2 in different Availability Zones
- Security group: alb-sg
- Listener: HTTP:80 forwards to tg1

Step 6: Test through the ALB DNS name

- : prove the application works through the load balancer.

Result: the page opened at loadbalancer-258149073.ap-south-1.elb.amazonaws.com and showed private IP 172.31.7.140, instance i-094bc2fe910454a3e, zone ap-south-1b.



Welcome to my PHP Web Application

Show Instance IP Address

Private IP: 172.31.7.140

Public IP: 13.234.67.121

Instance ID: i-094bc2fe910454a3e

Availability Zone: ap-south-1b

Part 3: Auto Scaling

Step 7: Create the launch template (mytemp)

- : a blueprint that tells the ASG how to build new instances.

- AMI: Amazon Linux 2023
- Instance type: t2.micro or t3.micro
- Security group: web-sg

- User data: the same script as Step 2

Step 8: Create the Auto Scaling Group (autoscaling)

- : automatically keeps between 1 and 3 instances running.

- Launch template: mytemp
- Subnets: same as the ALB
- Group size: Min 1, Desired 1, Max 3

Step 9: Attach the load balancer to the ASG

- : new instances register with the target group automatically.

- Load balancing: attach the existing target group tg1
- Health checks: enable ELB health checks, grace period 180 seconds

Step 10: Attach the existing instance to the ASG

- : the manually launched instance becomes a managed member of the group.

- The group was created with Desired 0, then the instance was attached using Actions, Instance settings, Attach to Auto Scaling Group.
- Then the group size was set to Min 1, Desired 1, Max 3. (The console rejects Min 1 while Desired is 0, so the instance must be attached first.)

Step 11: Add a target tracking scaling policy

- : add instances when average CPU goes above 50%, and remove them when it falls.

- Policy type: Target tracking scaling
- Metric: Average CPU utilization
- Target value: 50
- Instance warm-up: 120 seconds
- AWS creates the CloudWatch alarms AlarmHigh and AlarmLow automatically.

Part 4: Verify the scaling policies

Step 12: Generate CPU load

- : push CPU close to 100% artificially.

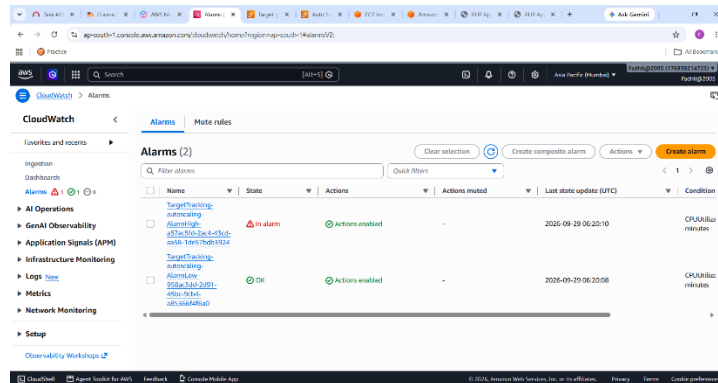
Connect with EC2 Instance Connect and run:

```
for i in $(seq $(nproc)); do yes > /dev/null & done
```

(Alternative: `sudo stress-ng --cpu 0 --cpu-method matrixprod --timeout 900s &`)

Step 13: Check the CloudWatch alarms

- : AlarmHigh going "In alarm" is the signal that triggers scale-out.

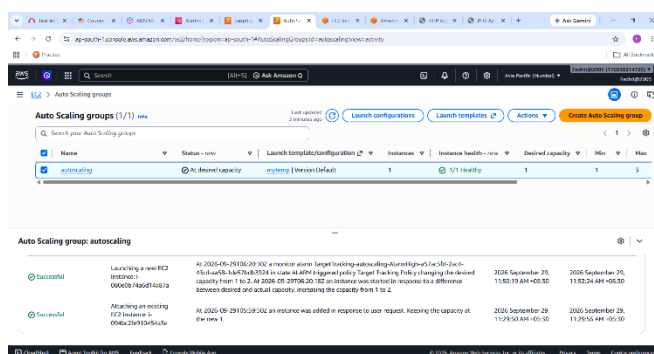


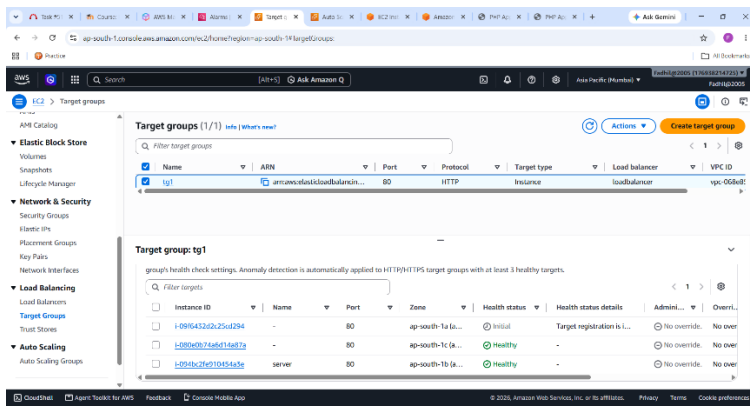
Result: TargetTracking-autoscaling-AlarmHigh changed to **In alarm** while AlarmLow stayed **OK**.

Step 14: Verify scale-out

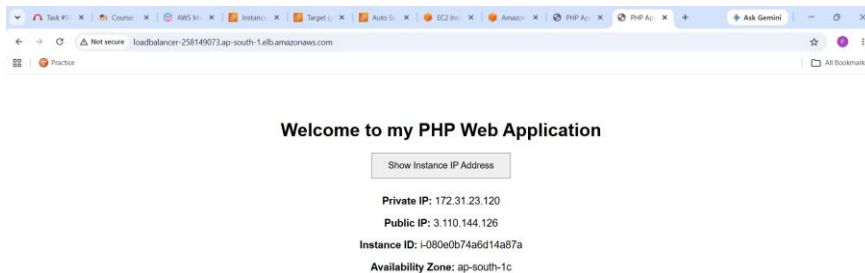
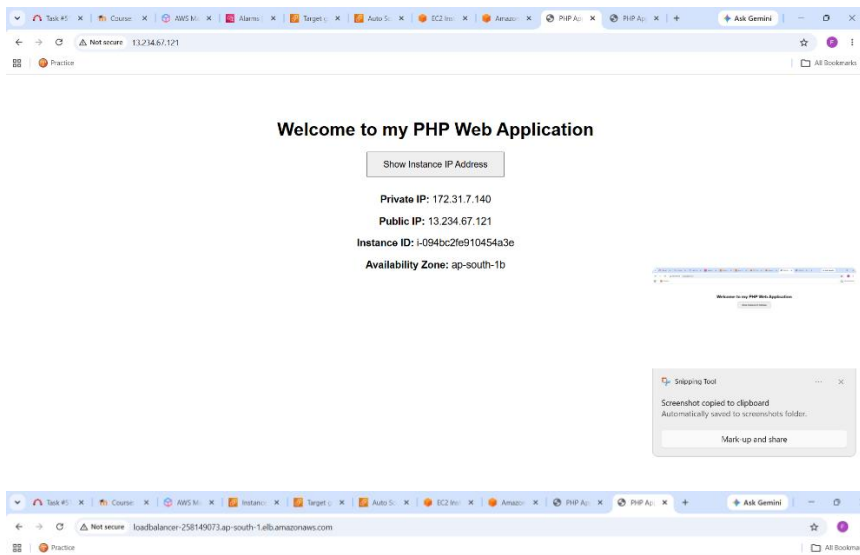
- : new instances launch and become healthy, which proves scale-out works.

- ASG, Activity tab: "Launching a new EC2 instance" entries
- Target group tg1, Targets: new instances change from initial to healthy
- ALB URL: refresh and click the button to see different private IPs and Availability Zones





When we host then refresh the page it shows



It shows two different ip

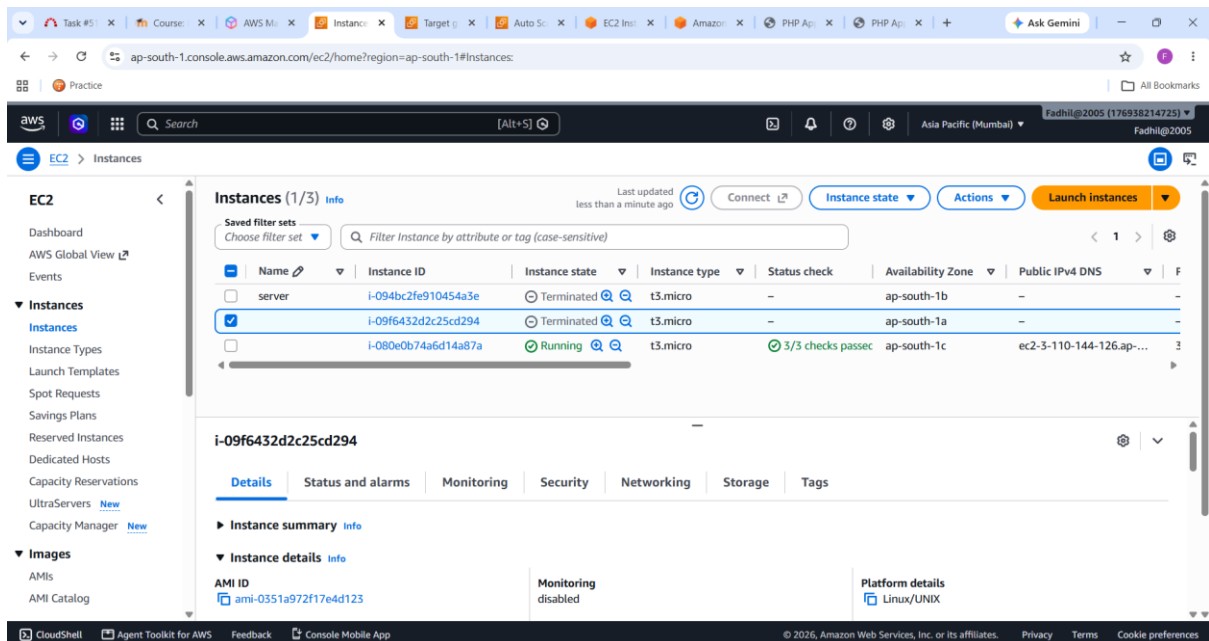
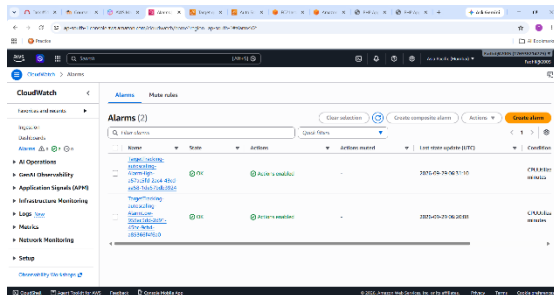
Step 15: Verify scale-in

- : removing the load makes the extra instances terminate, which proves scale-in works.

pkll yes

After about 15 minutes, AlarmLow goes into alarm and the Activity tab shows "Terminating EC2 instance" until the group returns to 1 instance.

Alarm goes of :



The two instance is terminated

Cleanup

Delete the ASG, the ALB, the target group, and any leftover instances to avoid charges.